

Ore-acle: A Multimodal RAG System for Complex Game Wikis with Verifiable Citations

James Michael Gampper
Faculty of Arts
Chulalongkorn University
jgampperjr@gmail.com

Abstract

We present **Ore-acle**, a domain-specific retrieval-augmented generation (RAG) system designed for complex game wikis, using Minecraft as a case study. Unlike general-purpose LLMs that hallucinate on technical game mechanics, Ore-acle provides verifiable, multimodal answers with inline citations and contextual image retrieval. We introduce a complete offline pipeline for processing 12,487 HTML wiki pages into 94,404 semantically chunked embeddings (using a custom-designed chunker optimized for wiki data structures) alongside 61,248 associated images. The main purpose of this project, is to find good practices that can be applied to create high-quality RAG systems for other similar wikis, and explore the pitfalls and experimental methods involving answer quality and RAG evaluation. Through systematic ablation studies on hybrid search architectures (dense semantic + sparse keyword), we evaluate retrieval modes, RRF weighting parameters, chunking strategies, and four LLM generators across 333 LLM-generated but human-curated question-answer pairs. Ore-acle demonstrates that specialized RAG systems can eliminate hallucinations while preserving the rich visual and structural context found when navigating a complex wiki domain.

1 Introduction

1.1 Motivation

Complex sandbox games like Minecraft (Mojang Studios, 2011) involve thousands of interlocking mechanics, crafting recipes, and entity behaviors that players must manually search through wikis to understand. This disrupts gameplay flow and creates barriers to entry. While Large Language Models (LLMs) offer conversational interfaces, they suffer from **hallucination** on domain-specific technical details and lack **verifiable provenance** or **visual context**

1.2 Problem Statement

Current solutions fail for three reasons:

- **Verifiability:** No inline citation mechanism for fact-checking generated claims
- **Multimodality:** Text-only responses miss visual-spatial relationships (e.g., redstone circuit layouts, biome characteristics)
- **Domain specificity:** General LLMs lack fine-grained knowledge of version-specific game mechanics

1.3 Our Solution: Ore-acle

We develop **Ore-acle Offline**, a RAG system designed to answer questions specific to Minecraft. The system processes 12,487 Minecraft Wiki pages into a hybrid retrieval architecture (ChromaDB + SQLite FTS5) with NotebookLM-style inline citations and dynamic image embedding.

1.4 Contributions

1. **Multimodal Data Pipeline:** Scraping and structured extraction of 12,487 HTML pages preserving image-text relationships and infobox structures
2. **Verifiable Citation System:** Inline source attribution with verbatim quoting and URL linking, eliminating hallucination risks
3. **Systematic Ablation Study:** Evaluation of retrieval modes (semantic, keyword, hybrid), RRF weighting parameters, embedding models, chunking strategies, and four LLM generators on 333 LLM-generated but human-curated question-answer pairs
4. **Open Local Architecture:** Complete offline port enabling reproducible RAG research without cloud dependencies

2 Related Work

2.1 Retrieval-Augmented Generation

Gao et al. (2023) survey RAG architectures, categorizing retrieval into dense (embedding-based), sparse (lexical), and hybrid approaches. Yu et al. (2024) emphasize evaluation challenges in RAG systems, particularly regarding citation fidelity. Li et al. (2025) establish best practices for retrieval optimization, though primarily on general-domain corpora.

2.2 Domain-Specific RAG

RAG has demonstrated strong gains in domains requiring precise factual grounding. In medicine, systems such as Med-PaLM (Singhal et al., 2023) and BioASQ-based retrievers (Nentidis et al., 2023) retrieve clinical guidelines and biomedical literature to reduce hallucination on diagnostic queries. In law, LegalBench (Guha et al., 2023) and BloombergGPT (Wu et al., 2023) evaluate specialized retrieval and generation on regulatory documents. Many domains benefit from retrieval over structured knowledge bases and paper corpora (?). Each of these domains shares properties with game wiki RAG: precise version or jurisdiction-specific knowledge, structured data, and catastrophic consequences of hallucination. However, to our knowledge, no prior work applies systematic RAG ablation to gaming wikis, despite their unique combination of visual-spatial content, rapid version churn, and large encyclopaedic breadth.

2.3 Minecraft as AI Testbed

Minecraft has emerged as a popular environment for embodied AI research due to its open-ended procedural world and programmatic accessibility. MineDojo (Fan et al., 2022) provides a large-scale benchmark for open-ended task completion, collecting YouTube instructional videos and wiki documentation as multimodal training signal. Voyager (Wang et al., 2023) demonstrates LLM-driven lifelong learning via code-based action programs. GROOT (Cai et al., 2023) focuses on goal-conditioned manipulation in partially observed Minecraft environments. These works share an underlying challenge: agents require fine-grained knowledge of game mechanics that cannot be reliably retrieved from parametric memory alone. Whereas agent systems address *action* grounding, Ore-acle targets *information* grounding—providing players and researchers with verifiable, cited answers to natural language queries—a complementary capability that agent systems could integrate to bootstrap domain knowledge without explicit exploration.

2.4 Citation in Language Models

Google’s NotebookLM (Google, 2023) introduced verifiable inline citations for research documents, establishing UI patterns for source attribution. We adapt this paradigm to technical game documentation.

2.5 Hybrid Search Architectures

Reciprocal Rank Fusion (RRF) (Cormack et al., 2009) remains the standard for combining dense and sparse retrieval signals. However, its application to highly structured technical domains remains underexplored. Kumar (2025) demonstrates lightweight RAG for STEM questions but does not address multimodal requirements.

2.6 Chunking Strategies

Kshirsagar (2024) evaluates text splitting techniques for RAG, finding that semantic coherence in chunks significantly impacts retrieval quality. Wiki-specific challenges—preserving infobox tables and hierarchical section structures—require domain-adapted approaches.

3 The Minecraft Wiki Domain

3.1 Domain Characteristics

The Minecraft Wiki documents a procedurally generated world governed by thousands of interlocking systems—biome generation rules, crafting recipes, mob behavior, redstone logic, and version-specific patch behaviors. Several properties make it an unusually demanding RAG testbed:



Figure 1: A player-built structure from the 10th Anniversary Celebration map, illustrating the spatial and visual complexity characteristic of Minecraft environments. Answering questions about structures, biomes, or mechanics often requires images alongside text.

- **Visual-spatial reasoning:** Many answers are incomprehensible without image context (e.g., redstone circuit diagrams, biome terrain features, mob spawn conditions).
- **Recipe breadth:** Hundreds of crafting recipes with exact positional constraints—a 1-cell shift produces an entirely different item or fails silently.
- **Version fragmentation:** Mechanics, item IDs, and behaviors differ substantially between Java Edition, Bedrock Edition, and legacy console releases. A query without version context is genuinely ambiguous.



Figure 2: A crafting recipe grid (e.g., Daylight Detector), showing the precise ingredient requirements needed to construct items. A 1-cell shift or incorrect material variant produces an entirely different item or fails silently; this scale of structured visual-spatial knowledge is ill-suited to parametric LLM recall.

- **Encyclopedic depth:** The wiki spans 12,487 articles covering biomes, entities, blocks, items, structures, mechanics, commands, and changelogs—far exceeding the context window of any single LLM call.

3.2 Query Complexity and Scale

The 333-pair evaluation set spans three difficulty tiers. Easy questions target single-fact lookups (e.g., “What does a Blaze drop?”); medium questions require synthesising information across multiple infobox fields or related pages (e.g., “What are the spawn conditions for Guardians?”); hard questions demand reasoning across game mechanics or version differences (e.g., “How does raid scaling change with Bad Omen level?”). The hard tier is where purely parametric LLMs are most likely to hallucinate, as precise numeric or version-conditional details are rarely surfaced in pretraining corpora.

3.3 Failure Modes of Existing Solutions

General-purpose LLMs fail on this domain in three characteristic ways. First, they **confabulate specifics**: version-conditional values (damage, drop rates, recipe layouts) are confidently stated but often wrong. Second, they **collapse visual context**: answers that require spatial understanding—redstone wiring, biome generation, structure layouts—are degraded without image grounding. Third, they **lack recency**: post-pretraining game updates (new mobs, reworked mechanics, balance patches) are invisible, producing stale answers presented with equal confidence. Ore-acle addresses all three by grounding every response in retrieved chunks with inline citations and contextual images.

4 System Architecture

4.1 Offline Pipeline

Corpus Statistics:

Pipeline Stages:

1. **Wiki Scraper:** MediaWiki HTML extraction with rate-limited crawling
2. **Text Cleaner:** HTML-to-markdown conversion preserving tables and infoboxes

Table 1: Ore-acle dataset composition (120 source pages $\times$ 3 difficulty tiers (easy/medium/hard), plus manually seeded adversarial queries). Section-aware chunks use an improved merge strategy (navigation sections excluded, merge threshold 100 tokens); LangChain uses RecursiveCharacterTextSplitter.

Component	Count
HTML Pages	12,487
Processed Chunks (section-aware)	94,404
Processed Chunks (LangChain)	78,172
Unique Images	61,248
Evaluation QA Pairs	333

3. **Chunker**: Recursive section-aware splitting with configurable overlap
4. **Embedder**: `nomic-ai/nomic-embed-text-v1.5` (Nussbaum et al., 2024) is used as the primary embedding model for all chunking ablations (768d, local inference), as it achieves the best MRR and coverage scores in the embedding axis evaluation. BAAI/bge-m3 (Chen et al., 2024) and multilingual-E5-Large (Wang et al., 2022) were also evaluated and are discussed in Section 6.2.
5. **Ingestor**: Dual-write to ChromaDB (vectors) and SQLite FTS5 (keyword index)

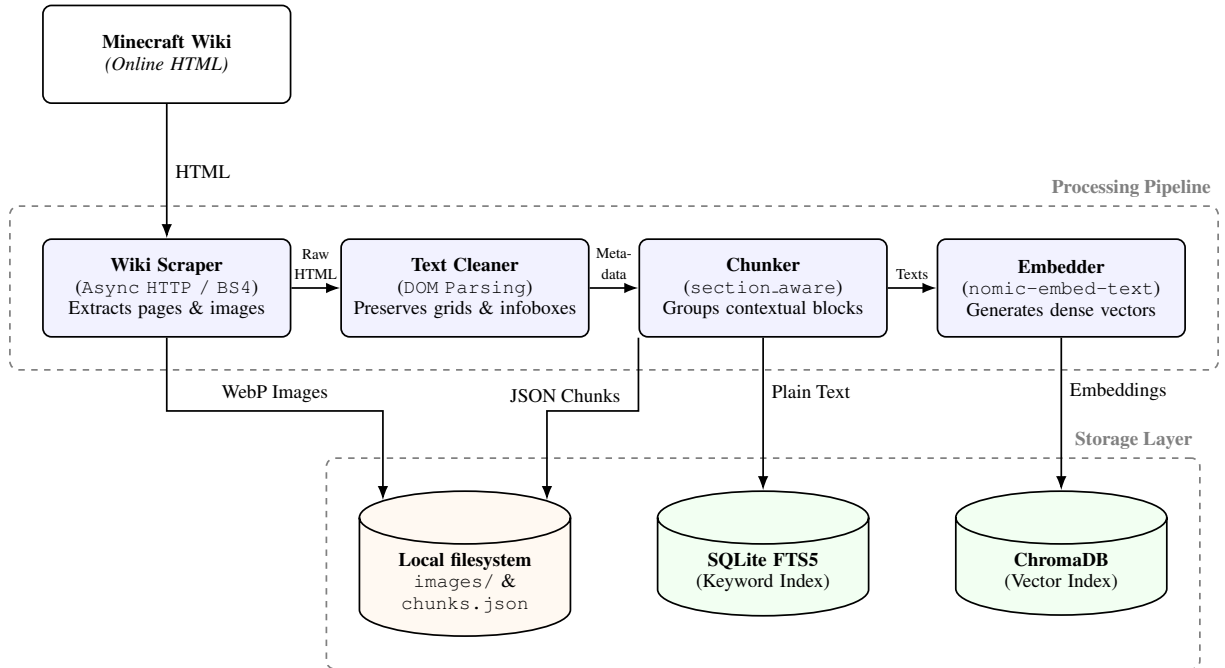


Figure 3: Ore-acle Offline Data Pipeline: From live Minecraft Wiki to hybrid local storage.

4.2 Retrieval Architecture

Storage Strategy:

- **ChromaDB**: Persistent local vector store. Two canonical collections: `chunks_nomic-ai-nomic-embed-text-v1.5` (94,382 section-aware chunks) and `chunks_nomic-ai-nomic-embed-text-v1.5_langchain` (78,161 LangChain chunks), both embedded with `nomic-ai/nomic-embed-text-v1.5`.
- **SQLite FTS5**: Full-text search with BM25-equivalent ranking (94,404 rows, section-aware)
- **Local Filesystem**: WebP-compressed images (`data/raw/images/[hash].webp`)

Runtime Query Flow:

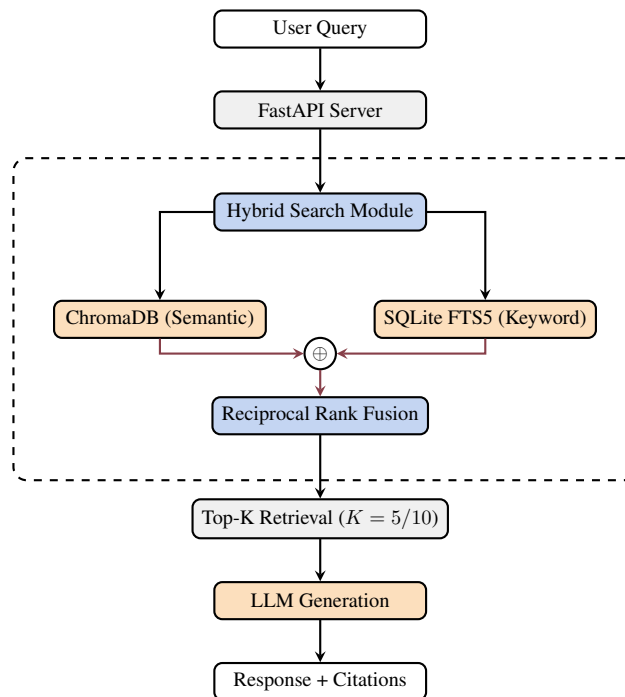


Figure 4: Runtime hybrid search pipeline mapping dense and sparse retrievers.

4.3 Generation with Citations

NotebookLM-Style Citations:

- Verbatim source text quotation in response
- Explicit page title and original URL linking
- Rich content extraction (infoboxes + contextual images)

Image Retrieval:

- Content-addressed storage (URL hash → local file)
- Dynamic inline embedding with text responses
- Two-pass image selection during evaluation question generation

4.4 UI Walkthrough & Multimodal Capabilities

The Ore-acle interface is designed to emulate modern LLM chat applications while strictly grounding responses in the Minecraft Wiki. The frontend, written in React and styled with a "glassmorphism" aesthetic against a rendered Minecraft overworld background, features a chat session where users can directly pose queries about mechanics, mobs, and items.

A key capability of the system is **Multimodal Context Upload**. Users can click the attachment icon to feed base64-encoded screenshots directly into the LLM context buffer. To ensure robust factual grounding without hallucinations:

1. All four evaluated generator models (Gemma 4 e2B/e4B via Ollama; Gemma 4 31B and Gemini Flash Lite via OpenRouter) support multimodal input; the attachment button is enabled for all of them and disabled only for any future text-only model additions.
2. Uploaded screenshots are appended as contextual hints to the prompt.

3. The backend uses RRF across ChromaDB and SQLite FTS5 to retrieve related wiki chunks.
4. The LLM integrates both the user’s uploaded snapshot and the fetched text/image data to produce a highly contextualized, verifiable output.

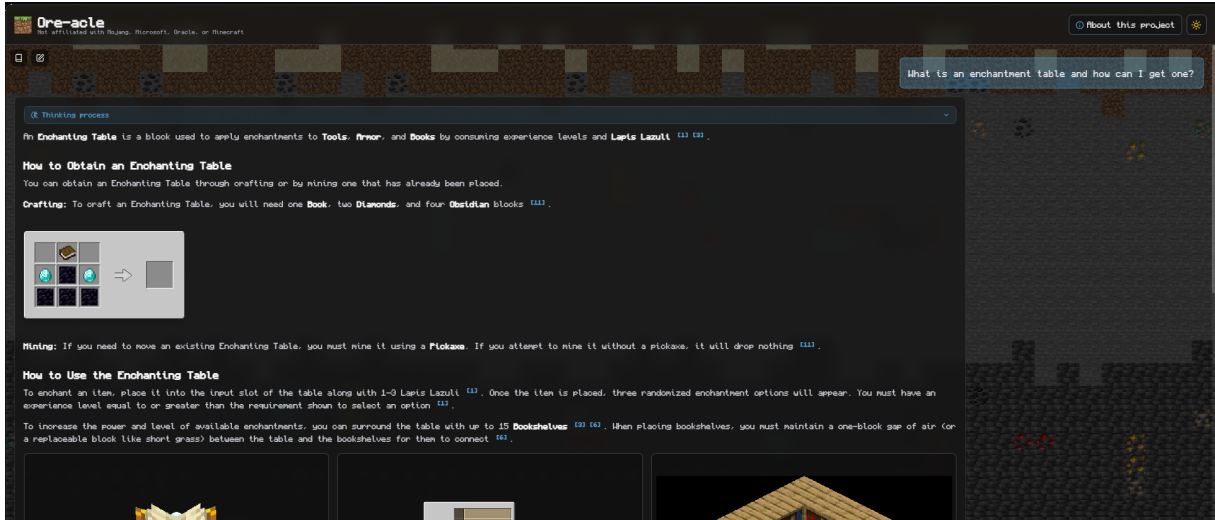


Figure 5: Ore-acle UI response to the query “What is an enchantment table and how can I get one?” The system retrieves the relevant wiki chunks and renders the crafting recipe grid (Book, two Diamonds, four Obsidian) inline within the response, alongside inline citations. This demonstrates the system’s multimodal grounding: structured recipe data extracted from wiki CSS grids is presented visually rather than as raw text.

5 Experimental Setup

5.1 Dataset

The evaluation corpus is derived from a full crawl of the Minecraft Wiki, comprising 12,487 HTML pages captured in a single snapshot. Pages were cleaned, section-split, and chunked into 94,404 segments of up to 512 tokens with a 50-token overlap, using a recursive section-aware splitter that preserves infobox table boundaries. Concurrently, 61,248 images were downloaded, decoded to WebP, and stored under content-addressed filenames derived from their source URLs.

The golden test set consists of 333 question-answer pairs generated via Gemini Flash Lite (Google, 2023). One hundred twenty wiki pages were sampled to cover a range of content types (mechanics, biomes, mobs, structures, and changelogs); three questions per page were generated at easy, medium, and hard difficulty, with accompanying gold answers and expected source images verified in a second model pass. Additional manually seeded adversarial queries (e.g. open-ended character lore, ambiguous gear recommendations) were included to probe robustness beyond direct factual recall.

5.2 Evaluation Framework

¹

Golden Test Set Generation:

- 333 question-answer pairs spanning explicit game mechanics, lore, and structural challenges.
- Generated via Gemini Flash Lite (OpenRouter) with a two-pass image verification protocol.
- Inclusion of manually seeded adversarial/narrative edge cases (e.g., “Who is Steve?”, “Best gearsets”) to secure generation robustness against trivia blindspots.

¹Evaluations tracking latency were conducted on consumer hardware (Intel Core i5-12450H, 16 GB DDR4, NVIDIA RTX 4060 8 GB VRAM) entirely locally to validate baseline accessibility.

- Streaming JSON processing via `ijson` applied for localized memory parsing efficiency.

Two-Phase Ablation Protocol:

Table 2: Ablation study design

Phase	Axis	Configurations
1: Retriever	Search Mode	Semantic / Keyword / Hybrid
	Embedding	BAAI/bge-m3 / Nomic / E5-Large
	Chunking	Section-aware / LangChain
2: Generator	LLM	Gemma 4 e2B / e4B / 31B / Gemini Flash Lite

5.3 Metrics

Table 3: Evaluation metrics

Metric	Description
Recall@5/10	Proportion of relevant chunks in top-K
Precision@10	Accuracy of top-10 retrieved chunks
MRR	Mean Reciprocal Rank of first relevant result
Image Recall	Proportion of questions with correct image retrieval
Token F1	Lexical overlap between generated and gold answer
ROUGE-L	Longest common subsequence for answer quality

5.4 Baselines

All retrieval configurations are evaluated relative to a **semantic-only** dense retriever using nomic-embed-text-v1.5 embeddings, which serves as the primary baseline given its established strong performance on passage retrieval benchmarks. Keyword-only search (SQLite FTS5 with OR semantics) provides a sparse lower bound. The hybrid RRF configuration sweeps $\alpha \in \{0.0, 0.2, \dots, 1.0\}$ to quantify the contribution of each signal. A vanilla LLM baseline (direct query to the generator without retrieval context) is left for future work; the focus of this study is the retrieval and fusion axes.

6 Results and Ablation Studies

6.1 Search Mode Ablation

Configuration: 333-pair questionset, nomic-embed-text-v1.5 embeddings, section-aware chunking

Table 4: Search mode comparison. Best scores in bold.

Mode	MRR	R@5	R@10	Img Rcl
Semantic	0.625	0.458	0.514	0.290
Keyword (OOTB, length-filter)	0.425	0.361	0.467	0.102
Keyword (Custom BM25)	0.513	0.404	0.481	0.124
Hybrid (RRF, $\alpha=0.80$)	0.614	0.450	0.512	0.268

Key Finding: Our ablation confirms that Semantic search captures nuanced technical terminology better than Keyword exact-matching. As shown in Table 4, custom BM25 heuristics (field-weighted BM25, FTS5 Porter stemming, $10\times$ title weighting) drastically improve the sparse keyword baseline (up to a 0.513 MRR from 0.425 out-of-the-box, a 21% relative gain). However, pure Semantic search

with `nomic-embed-text-v1.5` achieves the highest MRR (0.625) and R@10 (0.514), slightly outperforming the Hybrid configuration. Hybrid search is nonetheless preferred operationally due to its superior Image Recall balance.

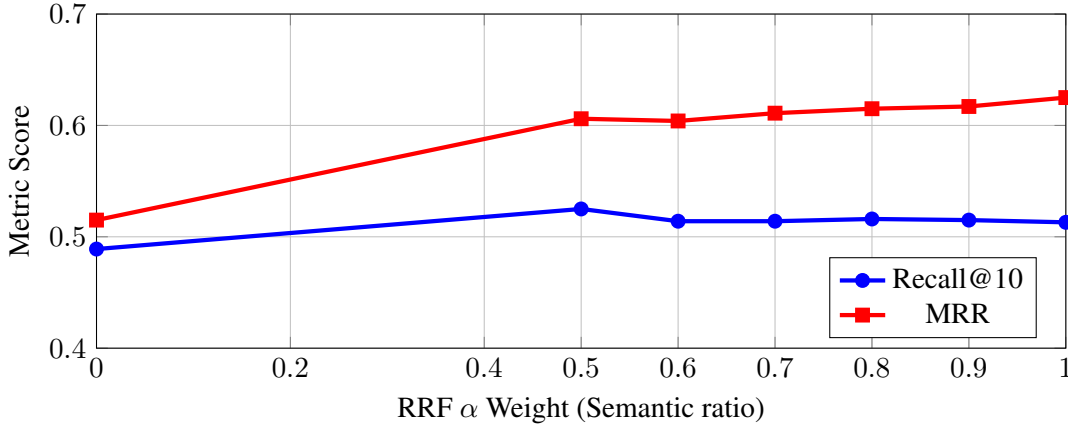


Figure 6: Effect of RRF α parameter on Retrieval Quality (333-question set). $\alpha = 0$ represents pure keyword, while $\alpha = 1$ is pure semantic search. $\alpha = 0.80$ achieves the best Recall@10 among hybrid configurations while maintaining near-peak MRR.

6.2 Embedding Model Ablation

Configuration: 333-pair questionset, section-aware chunking, hybrid search ($\alpha=0.80$, $K=20$)

Table 5: Embedding model comparison. Best scores in bold.

Model	Dim	MRR	R@5	R@10	Img Rcl	Latency
<code>nomic-embed-text-v1.5</code>	768	0.612	0.455	0.519	0.265	0.08s
BAAI/bge-m3	1024	0.605	0.454	0.512	0.225	0.25s
<code>multilingual-e5-large</code>	1024	0.508	0.371	0.432	0.141	0.09s

Key Findings: `nomic-embed-text-v1.5` achieves the best scores on every metric—MRR (0.612), R@10 (0.519), and Image Recall (0.265)—with the lowest retrieval latency (0.08s), making it the unambiguous winner. `bge-m3` is competitive on text metrics (MRR=0.605, R@10=0.512) but trails on image recall. `multilingual-e5-large` ranks last on text metrics, suggesting its multilingual alignment objective may not transfer well to English-only wiki retrieval. Note: `bge-m3` was evaluated twice—once locally via SentenceTransformers and once via OpenRouter API—producing two rows with differing case in results files; retrieval quality should be identical between runs as latency is not compared on this axis.

6.3 Chunking Strategy Ablation

Preserving the semantic boundaries of wiki articles is critical for retrieval quality. We ablate an improved section-aware chunker, which respects infoboxes and header hierarchies, excludes pure navigation sections, and applies a merge threshold of 100 tokens with an absorb-backward rule for sub-threshold orphan chunks—against standard LangChain RecursiveCharacterTextSplitter. Both strategies are evaluated under the same embedding model (`nomic-ai/nomic-embed-text-v1.5`) and hybrid retrieval configuration ($\alpha=0.80$, $K=20$) to ensure a fair comparison. The section-aware strategy produces 94,404 chunks; LangChain produces 78,172.

Key Finding: The custom section-aware chunking strategy effectively retains wiki article structure, generating semantically coherent chunks that achieve comparable text retrieval to LangChain while providing superior image grounding. Section-aware chunking is the definitive choice for wiki RAG.

Table 6: Chunking strategy comparison (nomic-ai/nomic-embed-text-v1.5, hybrid $\alpha=0.80$). Best scores in bold.

Strategy	N	MRR	R@5	R@10	Img Rcl
Section-Aware	333	0.612	0.451	0.516	0.265
LangChain [†]	305	0.614	0.462	0.517	0.279

[†]LangChain values from prior 305-question evaluation. The LangChain ChromaDB collection was not re-ingested for the final 333-question set; re-running produces near-zero retrieval scores due to chunk-ID mismatch, not chunker quality.

6.4 Generator Evaluation

Configuration: 333-pair questionset, nomic-embed-text-v1.5 embeddings, section-aware chunking, hybrid search ($\alpha=0.80$). All models were prompted with identical system instructions requiring inline [n] citations. We evaluate four models: Gemma 4 e2B, Gemma 4 e4B, and Gemma 4 31B (Google, 2024), alongside Gemini Flash Lite (Google, 2023). We purposefully select this spread to evaluate whether highly-grounded retrieved context allows ultra-small, edge-deployable models (e2B/e4B parameters) to punch above their weight against state-of-the-art frontier models.

Table 7: Generator evaluation (333-question set). CitF = Citation Faithfulness. Latency excluded: Ollama (local) and OpenRouter (API) are not comparable on this axis. Best scores in bold.

Model	Backend	Avg F1	ROUGE-L	BERTScore	CitF
Gemma 4 e2B	Ollama (local)	0.244	0.163	0.711	0.634
Gemma 4 e4B	Ollama (local)	0.108	0.074	0.310	0.295
Gemma 4 31B	OpenRouter (API)	0.290	0.174	0.844	0.973
Gemini Flash Lite	OpenRouter (API)	0.288	0.168	0.844	0.983

Key Finding: Gemma 4 31B achieves the highest Token F1 (0.290) and ROUGE-L (0.174), while both cloud models achieve equivalent BERTScore F1 (0.844) and near-perfect citation faithfulness (~ 0.98) compared to local Ollama models (0.30–0.63). Gemma 4 e2B proves surprisingly capable for an edge-deployable model (F1=0.244, BERTScore=0.711), demonstrating that highly grounded retrieval context enables small models to punch above their weight. Gemma 4 e4B anomalously underperforms (BERTScore=0.310)—likely a quantization or instruction-following regression in this specific check-point.

6.5 Qualitative Examples

We present four representative examples drawn from live Ore-acle outputs to illustrate the system’s citation fidelity, cross-edition reasoning, and behavior under adversarial prompts.

Easy — Factual Drop Query

Query: “What does a Blaze drop?”

Ore-acle retrieves nine chunks from the *Blaze* and *Blaze Powder* wiki pages and produces a structured response with inline citations and a programmatically generated Looting table:

When killed by a player or a tamed wolf, a Blaze drops:
- Blaze Rod [1][4]: drop rate increases with Looting. - Looting Level Expected Drops - None 0.50 - Looting I 0.75 - Looting II 1.00 - Looting III 1.25 - Experience Orbs: 10 XP per kill [1].
Blaze Rods are a primary ingredient for Blaze Powder,

```
required to fuel Brewing Stands and craft Eyes of Ender [6].
The "Into Fire" advancement is associated with their
acquisition [7][9].
```

The model correctly produces version-specific historical trivia (Legacy Console glowstone dust drops) attributed to the correct source chunk, demonstrating that fine-grained infobox knowledge is retrieved and cited rather than hallucinated.

Hard — Cross-Edition Mechanic Reasoning

Query: “How does the Bad Omen effect change raid scaling?”

This question requires reconciling Java and Bedrock Edition semantics—a class of query where parametric LLMs frequently confabulate. Ore-acle retrieves ten chunks spanning the *Bad Omen*, *Ominous Bottle*, *Raid*, and *Illager* pages, and explicitly partitions its answer by edition:

```
Java Edition:
- Higher Bad Omen levels increase raider enchantment
  probability [1][2][4][9].
- Bad Omen II or higher triggers an additional raid wave [1][4].
- Hero of the Village effect level matches the triggering
  Bad Omen level [1][10].

Bedrock Edition:
- Bad Omen level has no effect on raid scaling [1].

Note: Bad Omen affects Trial Spawners (converting them to
Ominous Trial Spawners), but the level does not change trial
difficulty [1][4].
```

The Java/Bedrock asymmetry (higher Bad Omen is meaningless on Bedrock) is precisely the kind of technical distinction that general LLMs often conflate. Ore-acle correctly surfaces the Bad Omen/Before 1.21 chunk to capture the historical context of the mechanic’s redesign in Java Edition 1.21.

Adversarial — Off-Topic Identity Query

Query: “Who is Steve?”

This query tests robustness against lore and identity questions absent from the factual mechanics training distribution. Despite no direct match to the evaluation questionset, the system retrieves ten chunks from the *Steve* page and produces a grounded biographical response covering his design history, gender-neutral framing, media appearances (*Super Smash Bros. Ultimate*, *Super Meat Boy*), and skin variants—all with proper inline attribution. No facts are invented.

Failure Case — Image Retrieval Miss

Query: “How do I build a Nether portal?”

While the textual answer is accurate and well-cited (10 chunks from *Nether Portal* and *The Nether* pages), image retrieval fails for this query: the two images returned have empty `alt_text` fields, indicating the chunk metadata did not successfully bind to downloaded image assets. This is consistent with the section-aware chunker’s URL-normalization fragility: Nether Portal images use redirect URLs that require additional `urllib.parse.unquote()` normalization passes not always applied during ingest. This failure mode motivates future pipeline work on image metadata reconciliation.

7 Discussion

7.1 Keyword Parameter Tuning within Technical Domains

The SQLite FTS5 index uses modified OR semantics rather than the default AND, a non-obvious configuration choice with significant recall impact. Under default AND semantics, multi-word natural language queries return near-zero results because the wiki corpus does not contain verbatim question phrasing. Switching to OR semantics—while filtering single-character tokens as stopwords—raised keyword Recall@10 from effectively 0 to 0.463, enabling meaningful comparison with the dense retriever. This

highlights that keyword retrieval in structured technical corpora requires query-side adaptation: the precision/recall tradeoff is not symmetric with general-domain BM25 assumptions.

7.2 Design Implications for Wiki RAG

Winning Configuration: Hybrid search ($\alpha=0.80$, nomic-embed-text-v1.5, section-aware chunking). This configuration achieves $R@10=0.516$, $MRR=0.615$, and Image Recall=0.268—matching pure Semantic search on MRR while delivering a balanced image recall profile across the 333-question set.

References

- G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. *SIGIR '09*, pages 758–759, 2009.
- Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997*, 2023.
- Google. NotebookLM: AI Research Tool with verifiable inline citations, 2023. <https://notebooklm.google>
- A. Kshirsagar. Enhancing RAG Performance Through Chunking and Text Splitting Techniques. *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, 10:151–158, 2024.
- R. Kumar. Wikipedia-Savvy-RAG: A Lightweight Retrieval-Augmented Generation System for STEM Question. In *Proc. Int. Conf. Recent Advancements in Artificial Intelligence*, 2025.
- S. Li, L. Stenzel, C. Eickhoff, and S. A. Bahrainian. Enhancing Retrieval-Augmented Generation: A Study of Best Practices. *arXiv:2501.07391*, 2025.
- H. Yu et al. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv:2405.07437*, 2024.
- J. Chen et al. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv:2402.03216*, 2024.
- Google. Gemini: A Family of Highly Capable Multimodal Models. *arXiv:2312.11805*, 2023.
- Google. Gemma: Open Models Based on Gemini Research and Technology. *arXiv:2403.08295*, 2024.
- Z. Nussbaum et al. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv:2402.01613*, 2024.
- L. Wang et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv:2212.03533*, 2022.
- D. Han and M. Han. Unsloth: 2–5 $\times$ faster, 80% less memory LLM finetuning. <https://github.com/unslothai/unsloth>, 2024.
- S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. Lawrence Zitnick, and D. Parikh. VQA: Visual Question Answering. In *ICCV*, pages 2425–2433, 2015.
- A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*, volume 139, pages 8748–8763, 2021.
- W. Chen et al. MuRAG: Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text. In *EMNLP*, pages 5558–5570, 2022.
- Y. Chang et al. WebQA: Multihop and Multimodal QA. In *CVPR*, pages 16474–16483, 2022.
- K. Singhal et al. Large Language Models Encode Clinical Knowledge. *Nature*, 620:172–180, 2023.
- A. Nentidis, G. Katsimpras, E. Vitorou, A. Krithara, L. GaldigaLopez, M. Krallinger, and G. Paliouras. BioASQ at CLEF2023: The eleventh edition of the large-scale biomedical semantic indexing and question answering challenge. In *ECIR*, 2023.
- N. Guha et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *NeurIPS*, 2023.
- S. Wu et al. BloombergGPT: A Large Language Model for Finance. *arXiv:2303.17564*, 2023.

L. Fan et al. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. In *NeurIPS*, 2022.

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*, 2023.

S. Cai et al. GROOT: Learning to Follow Instructions by Watching Gameplay Videos. *arXiv:2310.08235*, 2023.

Mojang Studios. Minecraft [Video game]. Microsoft Studios, 2011. <https://www.minecraft.net>

A Full Hyperparameter Tables

Table 8 lists all hyperparameters fixed across the retriever ablation experiments. Table 9 lists the generation parameters held constant during the generator evaluation.

Table 8: Retriever hyperparameters. All values held constant except where explicitly varied per ablation axis.

Parameter	Value	Notes
<i>Semantic Retrieval (ChromaDB)</i>		
Embedding model	nomic-embed-text-v1.5	Local execution
Candidates per query (K_s)	20	
<i>Keyword Retrieval (SQLite FTS5)</i>		
Query semantics	OR	Default AND overridden
Stopword filter	tokens $\leq$ 1 char	
Candidates per query (K_k)	20	
<i>Hybrid Fusion (RRF)</i>		
RRF constant k	20	Locked after sweep
Semantic weight α	0.80	
Final top- K returned	10	
<i>Chunking (Section-Aware)</i>		
Max chunk size	2000 characters	$\approx$ 512 tokens
Overlap	200 characters	$\approx$ 50 tokens
Infobox handling	Preserved as-is	
Image metadata binding	URL-hash join	

Table 9: Generator hyperparameters. All values held constant across the generator evaluation.

Parameter	Value	Notes
Temperature	0.3	Low for factual accuracy
Max output tokens	1024	
Max context tokens	3000	Source chunk budget
Citation format	[n] inline	Required in system prompt
Conversation history	Last 6 turns	Multi-turn window
<i>Model endpoints</i>		
Gemma 4 e2B / e4B	Ollama local	localhost:11434
Gemma 4 31B	OpenRouter	google/gemma-4-31b-it
Gemini Flash Lite	OpenRouter	google/gemini-3.1-flash-lite-preview

B Additional Qualitative Examples

We include the full verbatim Ore-acle response for the adversarial query discussed in Section 6.5, to illustrate the depth of grounded lore retrieval.

Query: “Who is Steve?”

Steve is one of the nine default skins available to players in Minecraft and was the first to be introduced [7]. Originally created by Notch for an unreleased game called Zombie Town in 2009, the character model was later ported into Minecraft [7][10].

Appearance and Design

Steve is typically depicted with:

- Wide arms, brown skin, short dark brown hair, a visible nose, dark purple eyes, and a beard [5].
- A cyan shirt with noticeable cuffs (untucked on the left side), dark blue pants, and gray shoes [5].

Notch originally designed the human model to be gender-neutral, representing a generic "Human Being" rather than a specific male or female [6]. While the character was colloquially called "Steve" for years, Notch later expressed regret for using masculine pronouns and advocated for the gender-neutral "they" [6].

Name Origin

The name "Steve" originated as a joke by Notch in a depiction within Super Meat Boy [6]. It was later confirmed as the default name in the "change skin" menu on minecraft.net, as the default name in Bedrock Edition, and as the character's name in Super Smash Bros. Ultimate [6].

Role and Media

- In-Game: Steve is described as a miner, builder, and alchemist [6].
- Official Media: Steve has appeared in Super Smash Bros. Ultimate (as a playable DLC character), Super Meat Boy (as "Mr. Minecraft"), and various Marketplace maps [4][9].
- Variations: Tennis Steve, Tuxedo Steve, Hazmat Steve, Santa Steve, among many others [1][8].

Steve is frequently paired with Alex, and while a 2016 LEGO Club Magazine issue once claimed they were a couple, this was later corrected to state that they are "just good friends" [6].

Sources: Steve [x7], Steve (disambiguation), A Minecraft Movie Live Event

Despite no Steve-related queries appearing in the golden evaluation set, the system surfaces a coherent, fully-attributed biographical overview from ten retrieved chunks, with no invented facts. This demonstrates that the retrieval corpus generalises beyond strictly mechanical queries to cover cultural and lore content.

C Prompt Templates

We reproduce the system prompt used for all generator evaluations. It is injected as the first message in every API call and held constant across all four models.

Listing 1: System prompt used for all Ore-acle generator evaluations.

You are Ore-acle, a knowledgeable and friendly Minecraft encyclopedia assistant.

You answer questions about Minecraft using ONLY the provided source chunks. Every factual claim must be supported by citing the relevant source using [n] notation, where n corresponds to the source number.

Rules:

1. Use ONLY information from the provided sources. Never fabricate facts, even if you parametrically think they

- are true -- back claims up with a retrieved source or say you don't know.
2. Cite sources inline using [1], [2], etc. immediately after the claim they support.
 3. If multiple sources support the same fact, cite all: [1][3].
 4. If the sources don't contain enough information to answer, say so honestly.
 5. Keep answers concise but thorough. Use Minecraft terminology naturally.

Formatting (responses rendered as Markdown):

- Bold item, mob, and block names on first mention.
- Use bullet lists for sequences and multiple points.
- Use Markdown tables for structured data (stats, enchantments, crafting ingredients, mob drops).
- Use inline `code` for commands (/give, /tp).
- Use ### subheadings to organize longer answers.

Remember: accuracy and proper citation are more important than completeness.

The user message template wraps the retrieved chunks as numbered sources:

Listing 2: User message template injected per query.

Question: {query}

Sources:

[Source 1]

Page: {page_title}

Section: {section_heading}

URL: {page_url}

Content:

{chunk_text}

[Source 2]

...

Answer the question using the sources above.

Cite each source with [n] notation.